

零知识证明论文阅读与研究设想

第一次组会汇报

徐诗霖

计算机科学与工程学院

2026 年 7 月



本次汇报想回答的问题

- 零知识证明到底解决了什么信任问题？
- 近期阅读的论文分别推进了哪一类能力？
- 从论文中学到哪些可迁移的研究方法？
- 是否能提出一个足够小、可实验验证的新方向？

汇报主线

从“证明一个结果正确”走向“证明一个 AI 系统按承诺、权限与策略执行”。

基本形式

证明者向验证者证明存在秘密见证 w ，使公开关系 $R(x, w) = 1$ 成立，但不泄露 w 本身。

$$\exists w : R(x, w) = 1$$

- x ：公开陈述，如承诺、输出、模型哈希、策略版本。
- w ：秘密见证，如输入、模型参数、中间计算 trace。
- π ：证明，验证者只需检查 $\text{Verify}(x, \pi) = 1$ 。

三条性质与能力边界

核心性质

- **完备性**：诚实执行应当通过验证。
- **可靠性**：错误声明难以伪造证明。
- **零知识性**：证明过程不泄露秘密。

需要保持清醒

- 不证明输入数据真实。
- 不证明模型公平、无偏或无后门。
- 不证明需求和规则本身合理。
- 证明的是“按给定规则正确执行”。

技术谱系：我目前的理解

技术路线	典型思想	优势	适合场景
Sigma/Schnorr	承诺--挑战--响应	简洁直观	身份认证
Groth16	电路特化 SNARK	证明极短	链上验证
PLONK	通用约束系统	工程生态好	通用应用
STARK	哈希与透明设置	无可信设置	大规模计算
GKR/Sumcheck	分层计算校验	适合矩阵/ML	ZKML、训练证明

- 我的切入点：不急着追所有协议细节，先理解“计算如何被表示、压缩和验证”。

已读论文地图

论文	关注问题	对我的启发
DIZK, USENIX Security 2018	分布式生成 zkSNARK	证明系统也是系统工程，瓶颈常在内存与并行化
zkCNN, CCS 2021	CNN 推理和准确率证明	卷积、ReLU、池化需要专门设计证明协议
Mystique, USENIX Security 2021	ZKML 中的类型转换与矩阵乘	真实 ML 不是单一算术电路，转换成本很关键
Kaizen, CCS 2024	深度网络训练证明	递归组合让多轮训练的证明保持简短
FAIRZK, arXiv 2025	公平性证明	可绕开逐样本推理，用模型参数与聚合统计证明性质
SAGA / CHaT, 2025–2026	Agent 安全治理与对抗	ZKP 可与身份、策略、审计结合，服务可信 Agent

DIZK: 把证明生成从单机推向集群

- 问题：传统 zkSNARK prover 是单机单体进程，内存随电路规模迅速膨胀。
- 方法：将证明生成拆分到计算集群，处理多项式、FFT、约束计算等并行任务。
- 结果：可扩展到十亿级逻辑门，论文报告比先前系统支持更大规模计算。
- 我的理解：ZKP 落地不只是协议正确，还要解决数据划分、通信、容错与资源调度。

Howard Wu et al., DIZK, USENIX Security 2018

zkCNN 与 Mystique: ZKML 的核心难点

zkCNN

- 面向 CNN 预测和准确率证明。
- 针对卷积、ReLU、MaxPool 设计交互式证明。
- 启发：利用 ML 结构，而不是直接套通用电路。

Mystique

- 关注算术/布尔、定点/浮点、承诺/认证值转换。
- 优化矩阵乘证明，面向大规模神经网络推理。
- 启发：转换层是系统性能的隐藏主战场。

Kaizen 与 FAIRZK：从推理正确到训练/性质正确

- **Kaizen**：证明模型确实按指定训练算法和数据承诺完成训练；递归组合使证明大小和验证时间不随迭代数增长。
- **FAIRZK**：证明模型满足公平性约束；关键不是逐样本证明每次推理，而是用模型参数与聚合信息构造公平性上界。
- 共同趋势：ZKP 正在从“证明一次计算输出”扩展到“证明模型来源、训练过程和高层性质”。

对我的启发

好的 ZK 应用往往不是把原任务硬塞进电路，而是重新寻找可证明、可压缩、可验证的等价表述。

Agent 安全论文给 ZKP 的新入口

- **SAGA**: 强调 Agent 身份、注册、访问控制 token、跨 Agent 通信策略。
- **Cloak, Honey, Trap**: 展示 LLM Agent 会被诱导、误导、循环和反制，安全边界非常脆弱。
- 共同问题: Agent 行为越来越自治，但用户、服务方和审计方很难确认它是否按承诺执行。

连接点

Agent 安全需要“可验证但不泄密”的执行证据；ZKP 可以把完整提示词、私有工具输入和业务数据藏在见证里，只公开合规证明。

阶段性心得：我学到的四件事

- ① **先定义可验证关系**：论文的核心常常藏在 $R(x, w)$ 怎么选。
- ② **避免天真电路化**：直接证明完整系统通常太慢，要找结构化捷径。
- ③ **区分证明目标**：推理正确、训练正确、公平性、权限合规是不同命题。
- ④ **工程指标同样重要**：证明时间、内存、证明大小、验证时间决定能否落地。

- 从应用痛点出发：谁不信任谁？秘密是什么？公开可验证的承诺是什么？
- 把系统拆成公开输入、秘密见证、中间 trace、验证关系四部分。
- 优先寻找“证明高层性质”的路径，而不是逐步证明每次底层计算。
- 做实验时先建最小原型：小模型、小策略、小数据集，跑通证明链路后再优化。

一句话

为 LLM Agent 生成“可验证执行证明”：证明一次 Agent 任务满足身份、策略、工具权限和关键状态约束，同时不泄露完整上下文。

- **密码学侧**：承诺、签名、Merkle 日志、零知识证明、可选递归聚合。
- **大模型安全侧**：越权工具调用、提示词泄露、恶意 Agent 协作、策略绕过。
- **核心命题**：我不要求你信任 Agent 的口头解释，而是验证它留下的密码学证明。

想证明什么

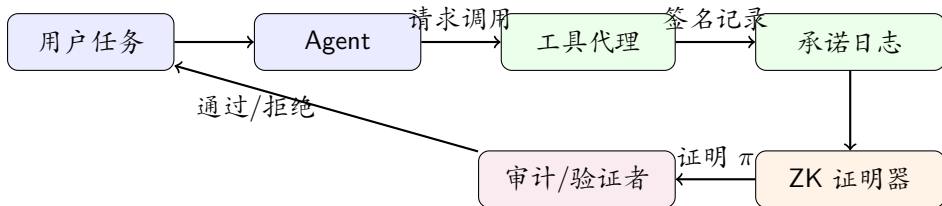
- Agent 使用了指定身份和策略版本。
- 每次工具调用都在授权集合内。
- 参数范围、调用顺序、次数满足约束。
- 输出绑定到同一份执行日志承诺。

不试图证明什么

- 不证明自然语言回答一定正确。
- 不证明大模型内部推理完全可靠。
- 不证明用户输入本身真实可信。
- 第一阶段只证明权限与流程合规。

- 对手：被 prompt injection 诱导的 Agent、恶意外部 Agent、试图伪造日志的执行环境。

ZK-AgentGuard: 系统草图



- 工具代理层强制记录并签名，避免 Agent 自己编造日志。
- 承诺日志作为公开锚点，完整提示词、工具返回和私有参数作为秘密见证。

可证明关系：把 Agent 行为电路化

形式化目标

$$R(\text{id}, H(P), H(L), y; L, \sigma, \text{trace}) = 1$$

- 公开输入：Agent 身份 id 、策略哈希 $H(P)$ 、日志承诺 $H(L)$ 、输出摘要 y 。
- 秘密见证：调用日志 L 、工具代理签名 σ 、敏感参数和中间 trace 。
- 电路检查：签名有效、Merkle 路径正确、每一步满足策略自动机转移、输出摘要与日志绑定。

为什么可落地

第一阶段只证明离散日志和策略，不证明 LLM 的每个 token 生成过程，电路规模可控。

最小可行实验设计

模块	第一版做法
场景	文件检索 Agent: 只能读授权目录, 不能外传敏感片段
策略 DSL	工具白名单 + 路径前缀 + 参数范围 + 调用顺序
日志机制	工具代理签名每次调用, Merkle tree 聚合日志
证明命题	日志中每一步满足策略, 最终输出绑定到日志根
原型工具	Noir 或 Circom 实现小电路, 先证明 10-100 步
评价指标	约束数、证明时间、验证时间、日志长度扩展性

预期成果

一个可演示 demo: 验证者看到输出、策略版本、日志根和证明, 但看不到完整用户输入与工具返回内容。

可能的难点与应对

- **策略语义**：自然语言安全策略难以直接证明。
应对：先定义结构化策略 DSL，再研究自然语言到 DSL 的可信编译。
- **日志可信性**：恶意 Agent 可能绕过或伪造日志。
应对：所有工具调用必须经过代理层，代理签名后才允许执行。
- **隐私与审计冲突**：审计需要证据，但日志包含敏感信息。
应对：日志做承诺，审计只验证零知识证明，必要时选择性打开片段。
- **性能压力**：长任务日志导致约束数增长。
应对：Merkle 承诺、分段证明、递归聚合，先做小规模基准。

下一阶段计划

- ① 补齐基础：Schnorr、Sigma 协议、Sumcheck/GKR、承诺方案。
- ② 深读两条线：Agent 治理安全与可验证计算/证明系统。
- ③ 完成 ZK-AgentGuard 的形式化定义：策略 DSL、公开输入、见证、关系、威胁模型。
- ④ 做最小原型：文件检索 Agent + 工具代理 + 日志承诺 + 合规证明。
- ⑤ 输出阶段成果：技术调研文档、demo、性能表格和局限性分析。

谢谢老师和同学们

欢迎批评指正